CSA0607 Design and Analysis of Algorithms

CO3 Assignment Report

Divide and Conquer: design, analysis and experimental evaluation

Role	Name	Register No.
Student A	__Rithika.G_____	_192571027_____

GitHub repository: https://github.com/192571027/CSA0607-

CO3_Assignment (pseudocode, code and outputs for each question are in the matching Q-folder).

All counts, tables and graphs in this report were produced by the programs in each question's Implementation_Code folder (fixed random seeds). Timings are machine dependent and are discussed as observations only.

1. Summary of Results

Q	Problem	Brute / standard	Divide and Conquer	Main experimental finding
1	Max/Min; Binary Search	2(n-1) comparisons; linear scan	T(n)=2T(n/2)+2 -> 3n/2-2; T(n)=T(n/2)+1 -> O(log n)	n=1024: 1534 vs 2046 comparisons; n=10^6 search needs at most 20 probes
2	Merge vs Quick Sort	-	Merge: Theta(n log n) always; Quick: O(n log n) average, O(n^2) worst	n=1023: depth 10 vs 1023 for Quick Sort depending on input; Merge depth 11 always
3	Karatsuba	n^2 digit multiplications	T(n)=3T(n/2)+cn -> n^1.585	n=2048: 4.19M vs 345K digit mults; faster in time only beyond ~512 digits (pure Python)
4	Strassen	n^3 multiplications	T(n)=7T(n/2)+cn^2 -> n^2.807	Fewer multiplications (n=128: 823,543 vs 2,097,152) but slower in every timed run here
5	Closest pair	n(n-1)/2 distances	T(n)=2T(n/2)+O(n) -> O(n log n)	n=4000: 4,637 vs 7,998,000 distance computations
6A	k-th smallest	QuickSelect, O(n^2) worst	Median of Medians: O(n) worst	n=2000, sorted input: 1,501,500 vs 11,290 comparisons (first-element pivot vs MoM)
6B	Subset sum	2^n subsets	Meet-in-the-Middle: 2^(n/2) time and memory	n=40: 2^40 subsets vs 2,097,152 sums generated

2. Q1: Max/Min and Binary Search

2.1 Algorithm design

Max/Min divides the array into two halves, solves each recursively to obtain (min, max) pairs and combines them with one comparison for the minimum and one for the maximum. Base cases are a single element (0 comparisons) and two elements (1 comparison). Binary Search divides a sorted array at the midpoint and discards the half that cannot contain the target.

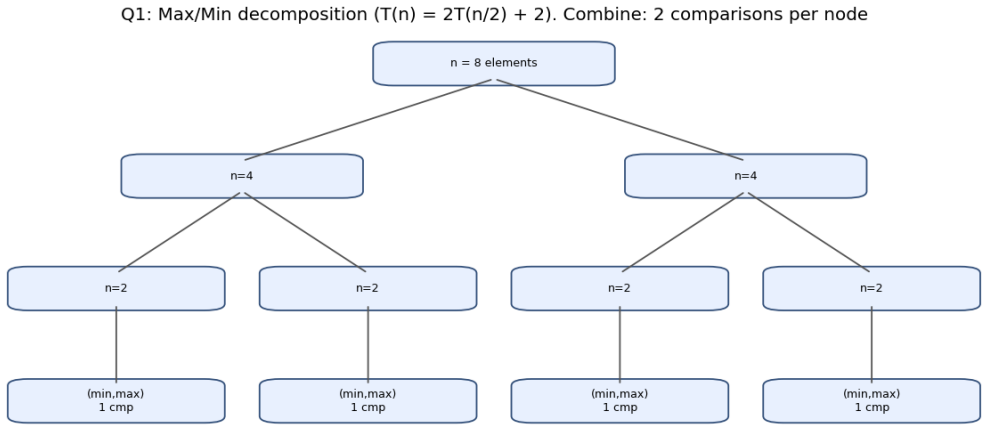


Figure 2.1: Max/Min decomposition for n = 8

```
MAXMIN(A, lo, hi):
 if lo == hi: return (A[lo], A[lo])
 if hi == lo+1: one comparison, return ordered pair
 mid = (lo+hi)/2
 (min1,max1) = MAXMIN(A, lo, mid);  (min2,max2) = MAXMIN(A, mid+1, hi)
 return (smaller(min1,min2), larger(max1,max2))     // 2 comparisons


BINARY_SEARCH(A, target):
 lo=0; hi=n-1
 while lo <= hi:
  mid=(lo+hi)/2
  if A[mid]==target return mid
  else if A[mid]<target lo=mid+1   // left half and mid eliminated
  else hi=mid-1                    // right half and mid eliminated
 return NOT_FOUND
```

2.2 Recurrence and complexity

- **Max/Min:** T(n) = 2T(n/2) + 2, T(2) = 1. For n = 2^k, unrolling gives T(n) = 2^(k-1) + 2(2^(k-1) - 1) = 3n/2 - 2. Two independent scans need 2(n-1), so the saving approaches 25%.
- **Binary Search:** T(n) = T(n/2) + 1, T(1) = 1, so T(n) = floor(log2 n) + 1 = O(log n). The interval length at least halves every iteration, and an empty interval terminates the loop.

2.3 Test datasets

Max/Min: random, sorted, reverse-sorted and all-equal arrays for n between 2 and 65,536 (results checked against Python's min and max). Binary Search: arrays of even numbers of size 1 to 10^6; every element searched (successful), every odd value plus values below the minimum and above the maximum (unsuccessful), plus first and last element.

2.4 Experimental results

n	D&C observed	ceil(3n/2) - 2	2(n-1)	Saving
8	10	10	14	28.6%
64	94	94	126	25.4%
100	162	148	198	18.2%
1024	1534	1534	2046	25.0%
10000	15902	14998	19998	20.5%
65536	98302	98302	131070	25.0%

Table 2.1: Max/Min comparisons. The count is identical for all four data patterns because the algorithm is data independent.

n	floor(log2 n)+1	Successful avg	Successful max	Unsuccessful avg	Unsuccessful max
8	4	2.62	4	3.30	4
1000	10	8.99	10	9.98	10
100000	17	15.69	17	16.69	17
1000000	20	18.96	20	19.95	20

Table 2.2: Binary Search probes. The theoretical bound was never exceeded.

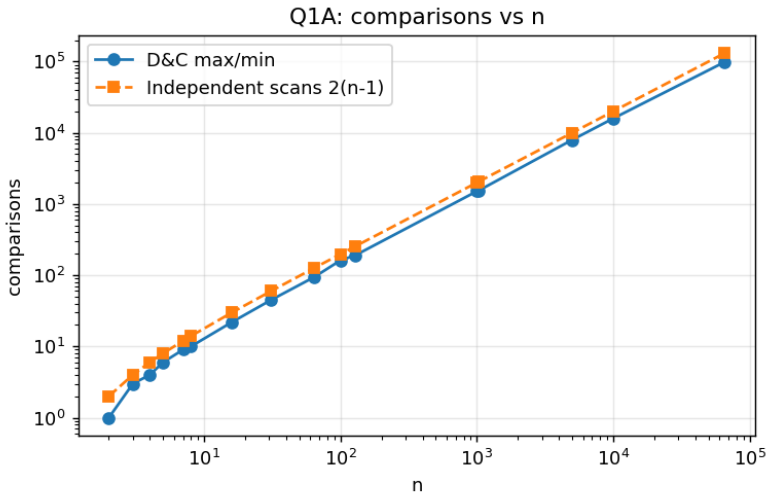


Figure 2.2: Max/Min comparisons versus n

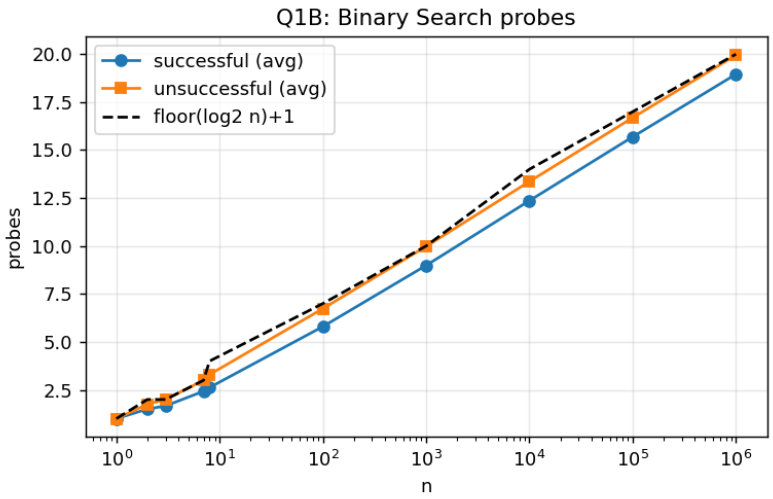


Figure 2.3: Binary Search probes versus n

2.5 Observations and conclusion

- The 3n/2 - 2 formula matches exactly when n is a power of two. For other n the observed count is higher (n = 100: 162 versus 148) because uneven halves create size-3 and size-5 subproblems whose combine step still costs two comparisons. The bound is attainable for every n by first comparing elements in pairs, then finding the maximum among the winners and the minimum among the losers.
- Binary search on 10^6 elements needs at most 20 probes, against up to 1,000,000 for a linear scan. Unsuccessful searches cost about one probe more than successful ones on average, because they always run to an empty interval.
- Divide and conquer reduces the constant in Max/Min by about 25%, but the asymptotic order stays Theta(n); for Binary Search the halving yields a genuine asymptotic improvement.

3. Q2: Merge Sort and Quick Sort

3.1 Algorithm design

Merge Sort always divides into equal halves and does the work in the combine step (merge). Quick Sort does its work in the divide step: partitioning around a pivot. The stated pivot strategy is the last element, with an out-of-place stable partition (L = elements less than the pivot, R = the rest). The quality of the decomposition therefore depends on the input.

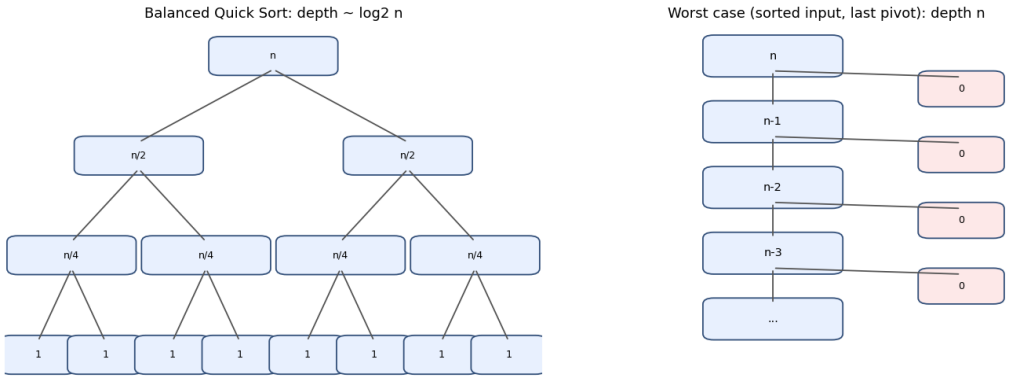


Figure 3.1: Balanced versus worst-case Quick Sort recursion

```
MERGE_SORT(A): if |A|<=1 return A
 L = MERGE_SORT(left half); R = MERGE_SORT(right half); return MERGE(L,R)


QUICK_SORT(A): if |A|<=1 return A
 p = A[last]
 L = {x in A[0..last-1] : x <  p};  R = {x in A[0..last-1] : x >= p}
 return QUICK_SORT(L) + [p] + QUICK_SORT(R)
```

3.2 Recurrence and complexity

- **Merge Sort:** T(n) = 2T(n/2) + n - 1 gives Theta(n log n) for every input (master theorem, case 2). Depth = ceil(log2 n) + 1.
- **Quick Sort, balanced:** T(n) = 2T(n/2) + n, so O(n log n).
- **Quick Sort, worst:** T(n) = T(n-1) + n - 1 = n(n-1)/2 = O(n^2). It occurs when the pivot is always the minimum or maximum, such as sorted input with a last-element pivot.
- **Constant ratio r : (1-r):** depth = log n / log(1/r), so total work stays O(n log n) for any fixed r, with a larger constant as r approaches 1.

3.3 Test datasets

n = 1023 for the main comparison. Balanced input was constructed so that the median sits last and each half is recursively arranged the same way, which forces a perfectly balanced partition at every level. Other

inputs: random permutation, constructed 90:10 skew, sorted (worst) and reverse-sorted. Scaling runs used n = 64 to 1500.

3.4 Experimental results

Input (n = 1023)	Comparisons	Max depth	First partitions (L|R)
Quick Sort, balanced	8,194	10	511|511, 255|255, 127|127
Quick Sort, random	12,206	24	197|825, 169|27
Quick Sort, 90:10 skew	15,562	42	919|103, 826|92
Quick Sort, sorted	522,753	1023	1022|0, 1021|0, 1020|0
Quick Sort, reverse sorted	522,753	1023	0|1022, 0|1021
Merge Sort (sorted input)	5,110	11	511|512 halves (fixed)

Table 3.1: Quick Sort partition investigation compared with Merge Sort.

Split ratio (n = 1000)	Max depth	Comparisons	Predicted depth log n / log(1/r)
50:50	10	7,987	10.0
70:30	16	8,865	19.4
90:10	41	15,154	65.6
99:1	202	70,972	687.3

Table 3.2: Effect of the split ratio. The formula is only an asymptotic estimate (it ignores that each pivot is removed and that small sub-arrays end early), so observed depths are lower.

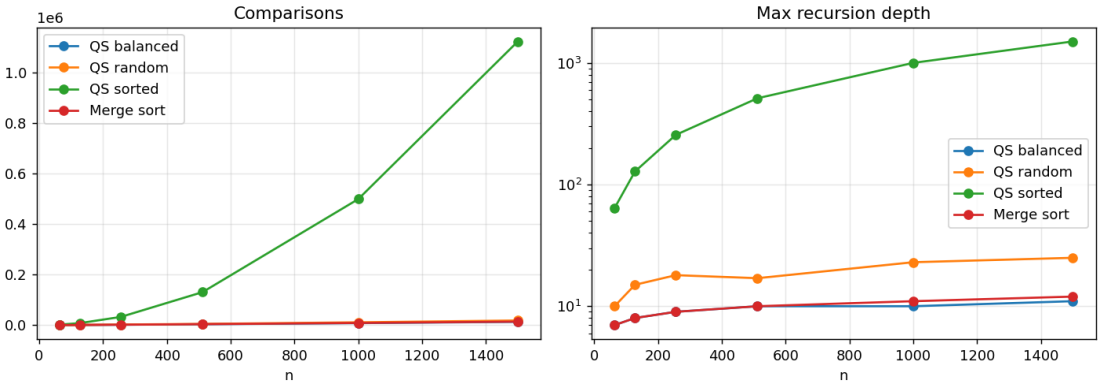


Figure 3.2: Comparisons and recursion depth for increasing n

3.5 Observations and conclusion

- The partition-size listing for sorted input (1022|0, 1021|0, ...) shows exactly why the recurrence degrades to T(n-1) + n and the depth reaches n. Reversed input fails in the mirrored way.
- Even a 90:10 split stays O(n log n) (15,562 comparisons) but needs about 4 times the depth of a balanced split and roughly twice the comparisons. Randomly ordered input behaves close to the balanced case.

- On random data of the same size Merge Sort used about 23% fewer comparisons than Quick Sort (n = 1000: 8,697 versus 11,272). Its recursion shape and Theta(n log n) bound are the same for every input (only the number of comparisons inside each merge varies, which is why the sorted-input figure of 5,110 is lower than for random data), at the cost of O(n) auxiliary space. Quick Sort is attractive when in-place partitioning, cache locality and a good pivot strategy (random or median-of-three) are available.
- **Joint design question:** before choosing, collect: input size, how sorted or duplicate-heavy the data is, whether adversarial inputs are possible, stability requirement, memory limits, comparison cost, array versus linked list, and the need for worst-case guarantees.

4. Q3: Karatsuba Multiplication

4.1 Algorithm design

Write x = a*B + b and y = c*B + d with B = 10^m, m = n/2. Then x*y = ac*B^2 + (ad + bc)*B + bd. The middle term is obtained from one product: (a+b)(c+d) - ac - bd = ad + bc. This reduces four half-size multiplications to three.

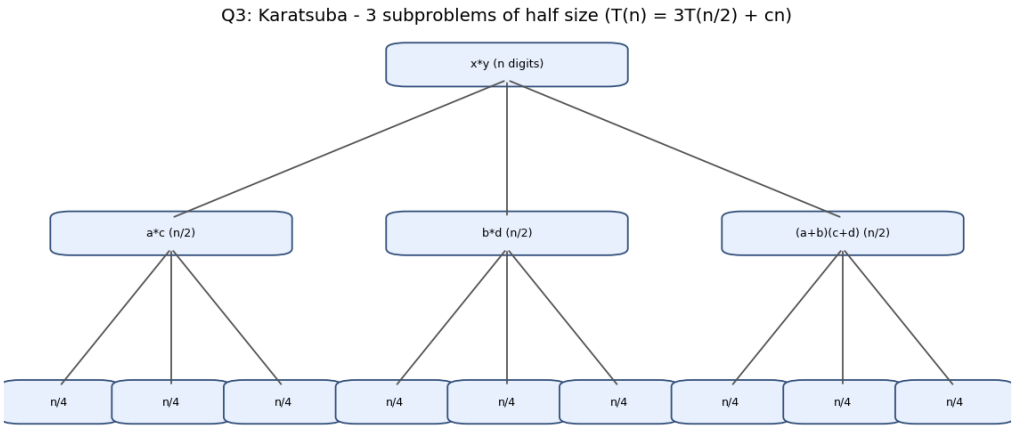


Figure 4.1: Karatsuba decomposition (three subproblems per level)

```
KARATSUBA(x, y):
 n = max(digits(x), digits(y));  if n == 1 return x*y
 m = n/2;  B = 10^m
 (a,b) = (x div B, x mod B);  (c,d) = (y div B, y mod B)
 z2 = KARATSUBA(a, c);  z0 = KARATSUBA(b, d)
 z1 = KARATSUBA(a+b, c+d) - z2 - z0
 return z2*B^2 + z1*B + z0
```

4.2 Recurrence and complexity

T(n) = 3T(n/2) + cn. In the recursion tree level i has 3^i subproblems of size n/2^i, costing (3/2)^i * cn per level. The geometric series is dominated by the leaves, so T(n) = Theta(3^(log2 n)) = Theta(n^(log2 3)) = Theta(n^1.585). The standard method is Theta(n^2) since T(n) = 4T(n/2) + cn gives n^(log2 4).

4.3 Test datasets

Random decimal numbers with 2, 4, 8, ..., 2048 digits; correctness was checked on 180 random pairs from 1 to 257 digits against Python's built-in multiplication. The cost model counts single-digit multiplications and also measures best-of-5 wall-clock time.

4.4 Experimental results

Digits n	Schoolbook mults	Karatsuba mults	Schoolbook ms	Karatsuba ms	Faster
16	256	151	0.038	0.108	Schoolbook

Digits n	Schoolbook mults	Karatsuba mults	Schoolbook ms	Karatsuba ms	Faster
64	4,096	1,405	0.524	1.053	Schoolbook
256	65,536	12,747	8.73	9.68	Schoolbook
512	262,144	38,293	37.80	29.17	Karatsuba
1024	1,048,576	115,059	156.4	87.9	Karatsuba
2048	4,194,304	344,759	673.6	270.7	Karatsuba

Table 4.1: Digit multiplications and time. Karatsuba's counts slightly exceed 3^k because a+b and c+d can have one extra digit.

Cutoff (digits)	1 (pure)	8	16	32	64	128
Time ms, n = 1024	100.1	40.0	32.7	67.2	45.5	53.8

Table 4.2: Hybrid Karatsuba (schoolbook below the cutoff); the fastest cutoff in this run was 16 digits (the optimum is noisy at this timer resolution).

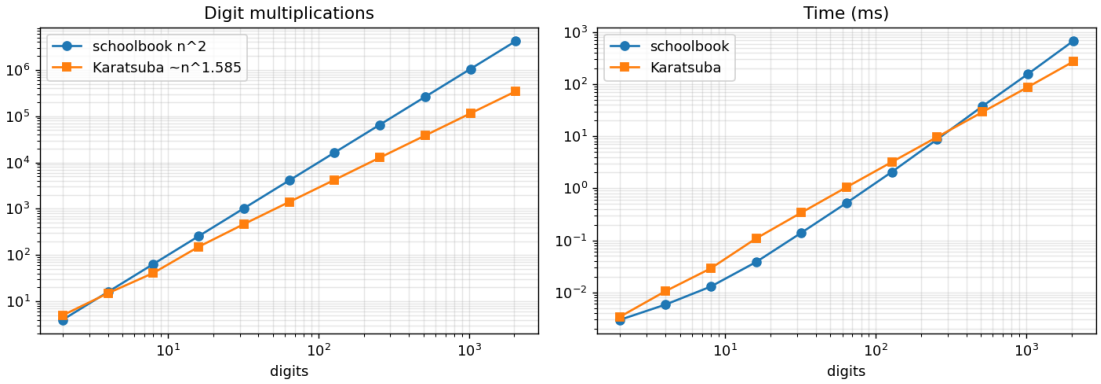


Figure 4.2: Multiplications and time versus number of digits

4.5 Observations and conclusion

- Multiplication counts favour Karatsuba from n = 4 onward, yet running time favours it only beyond roughly 256 to 512 digits. The extra additions, subtractions, splitting and recursion overhead dominate for small inputs.
- A hybrid with a base case of about 16 digits was about 3 times faster than pure Karatsuba at n = 1024, which is the standard engineering fix.
- Asymptotic complexity is a statement about growth, not about the fastest algorithm at every size; the crossover depends on hardware, language and implementation.

5. Q4: Strassen Matrix Multiplication

5.1 Algorithm design

Conventional block multiplication forms the four result blocks from eight half-size products. Strassen forms seven products M1..M7 using sums and differences of blocks and then recombines them.

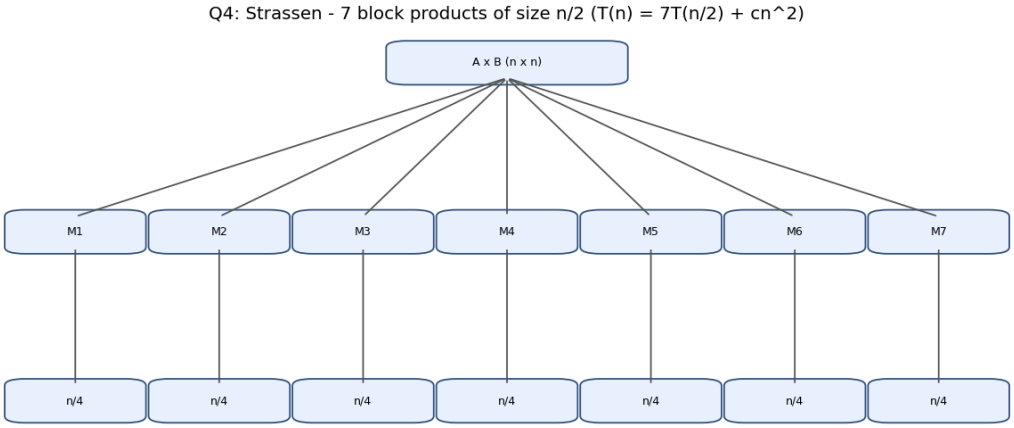


Figure 5.1: Strassen decomposition into seven half-size products

M1=(A11+A22)(B11+B22) M2=(A21+A22)B11 M3=A11(B12-B22) M4=A22(B21-B11)
M5=(A11+A12)B22 M6=(A21-A11)(B11+B12) M7=(A12-A22)(B21+B22)
C11=M1+M4-M5+M7 C12=M3+M5 C21=M2+M4 C22=M1-M2+M3+M6

Correctness check for two blocks: M3 + M5 = A11(B12 - B22) + (A11 + A12)B22 = A11B12 + A12B22 = C12. M2 + M4 = (A21 + A22)B11 + A22(B21 - B11) = A21B11 + A22B21 = C21. Expanding the remaining two sums gives C11 and C22. The program also checked exact equality against the conventional method on integer matrices of sizes 1 to 31. Non-power-of-two sizes are zero-padded to the next power of two and the result is cropped.

5.2 Recurrence and complexity

Conventional: T(n) = 8T(n/2) + O(n^2) = Theta(n^3). Strassen: T(n) = 7T(n/2) + cn^2, with a = 7 > b^2 = 4, so T(n) = Theta(n^(log2 7)) = Theta(n^2.807). Strassen performs 18 block additions per level.

5.3 Test datasets

Random integer matrices with entries in [-9, 9], sizes 2 to 128 (pure Python, operation counting); padding study for n = 33, 65, 100, 129, 200; NumPy float64 matrices of size 128 to 2048 against the optimised library multiply.

5.4 Experimental results

n	Conv mults	Strassen mults	Conv adds	Strassen adds	Conv ms	Strassen ms
8	512	343	448	1,674	0.06	1.29

n	Conv mults	Strassen mults	Conv adds	Strassen adds	Conv ms	Strassen ms
32	32,768	16,807	31,744	94,698	2.15	64.9
64	262,144	117,649	258,048	681,318	17.5	466.4
128	2,097,152	823,543	2,080,768	4,842,954	142.9	3,607.4

Table 5.1: Pure Python, Strassen with a base case of 1 element.

Base-case cutoff (n = 128)	Multiplications	Additions	Time ms
Conventional	2,097,152	2,080,768	147.9
1	823,543	4,842,954	3,597.1
8	1,229,312	1,899,328	236.0
32	1,605,632	1,758,208	138.7
64	1,835,008	1,880,064	137.0

Table 5.2: Effect of the cutoff. A large cutoff approaches the conventional method; the gain over conventional is marginal.

n	Padded size	Extra cells	n^3	7^log2(padded)
65	128	287.8%	274,625	823,543
100	128	63.8%	1,000,000	823,543
129	256	293.8%	2,146,689	5,764,801

Table 5.3: Padding overhead. For n = 65 and 129 Strassen on the padded matrix needs more multiplications than the conventional method on the original matrix.

n	BLAS matmul ms	Strassen (cutoff 128) ms	Ratio	Max abs error
256	0.97	3.11	3.2	1.1e-13
1024	38.3	129.2	3.4	8.9e-13
2048	287.0	1015.7	3.5	3.1e-12

Table 5.4: NumPy comparison.

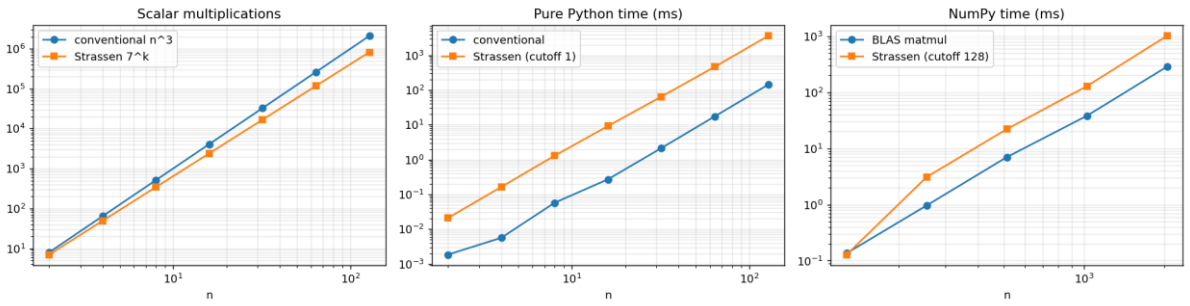


Figure 5.2: Multiplications, pure-Python time and NumPy time

5.5 Observations and conclusion

- Strassen reduces multiplications by a factor of 1.14 (n = 2) to 2.55 (n = 128), growing with n, but it more than doubles the additions and allocates many temporary matrices. In pure Python it was about 25 times slower at n = 128; with a cutoff of 64 it merely matched the conventional method.
- Against an optimised BLAS routine, which uses cache blocking and vectorisation, Strassen with an efficient base case was about 3 times slower for every n from 256 up to 2048 (and only on par at n = 128). The floating-point error was larger but small (about 10^-12).
- Recommendation: use conventional (library) multiplication for small and medium or non-power-of-two sizes and for numerically sensitive work; consider Strassen only for very large dense matrices with a tuned cutoff and enough memory.

6. Q5: Closest Pair of Points

6.1 Algorithm design

The brute-force method (CO2) compares all pairs. The divide-and-conquer method sorts the points by x once, splits at the median x, solves both halves recursively and then handles pairs that straddle the dividing line. Each call returns its best distance and pair, together with its points sorted by y so the combine step can merge them in linear time.

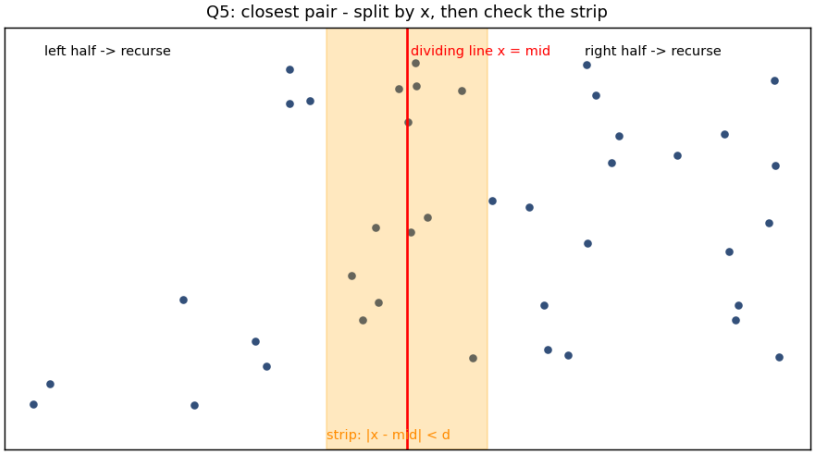


Figure 6.1: Split by a vertical line, then inspect the strip

```
CLOSEST(P):                  // P sorted by x; also returns points sorted by y
 if |P| <= 3: brute force
 (dL,YL) = CLOSEST(left);  (dR,YR) = CLOSEST(right);  d = min(dL, dR)
 Y = MERGE(YL, YR)
 strip = [p in Y : |p.x - midx| < d]
 for each p in strip: compare with following q while q.y - p.y < d   // at most 7 neighbours
 return best
```

The boundary requires special treatment because the closest pair may have one point in each half; neither recursive call sees such a pair. Within the strip, each point needs only its next few neighbours in y-order, because a d x 2d rectangle can hold at most eight points that are pairwise at least d apart.

6.2 Recurrence and complexity

T(n) = 2T(n/2) + O(n) (merge plus strip scan), so T(n) = O(n log n) with the x-sort done once. Brute force performs n(n-1)/2 distance computations = O(n^2).

6.3 Test datasets

Uniform random points (n from 10 to 100,000), five clustered groups (Gaussian), collinear horizontal points and points on one vertical line, each at n = 2000; 300 additional small random cases were checked against brute force.

6.4 Experimental results

n	Brute dists	D&C dists	Ratio	Brute ms	D&C ms
99	4,851	126	38.5	1.40	0.32
500	124,750	535	233	27.0	1.66
1000	499,500	1,138	439	151.7	3.81
2000	1,999,000	2,286	875	520.3	8.48
4000	7,998,000	4,637	1,725	2,139.6	16.9
100,000	4,999,950,000 (needed)	142,157	-	infeasible	1,153

Table 6.1: Distance computations and time on uniform random points.

Distribution (n = 2000)	Brute dists	D&C dists	D&C ms
Uniform	1,999,000	2,292	8.1
Clustered	1,999,000	2,134	8.4
Collinear (same y)	1,999,000	1,236	5.5
Same x	1,999,000	1,259	9.7

Table 6.2: Behaviour on different point distributions. The D&C method is correct and efficient in all four.

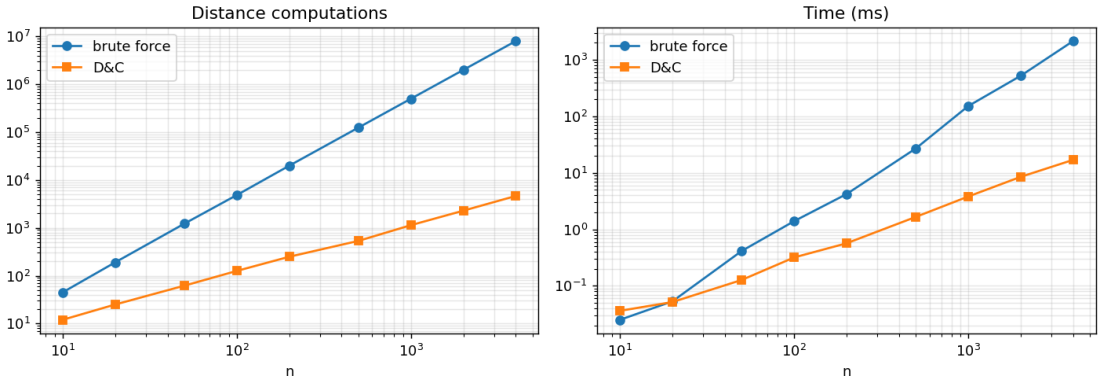


Figure 6.2: Distance computations and time versus n

6.5 Observations and conclusion

- D&C distance computations divided by n log2 n fall from 0.36 to about 0.10 and level off, consistent with O(n log n). Brute force is exactly n(n-1)/2.
- For very small inputs the brute-force method is faster in practice: it won for n up to 16 and D&C won from n = 32 in this implementation, because of recursion, sorting and merge overhead. A practical implementation therefore uses brute force for small base cases (3 here).
- Skipping the strip logic would miss straddling pairs; the strip's y-ordered scan is what keeps the combine step linear.

7. Q6: Advanced Divide and Conquer

7.1 Median of Medians (Student A primary)

Goal: find the k-th smallest element in worst-case linear time. The elements are divided into groups of five, the median of each group is found, and the median of those medians is used as the pivot. The array is then partitioned into elements less than, equal to and greater than the pivot, and the search continues in the one part that contains rank k.

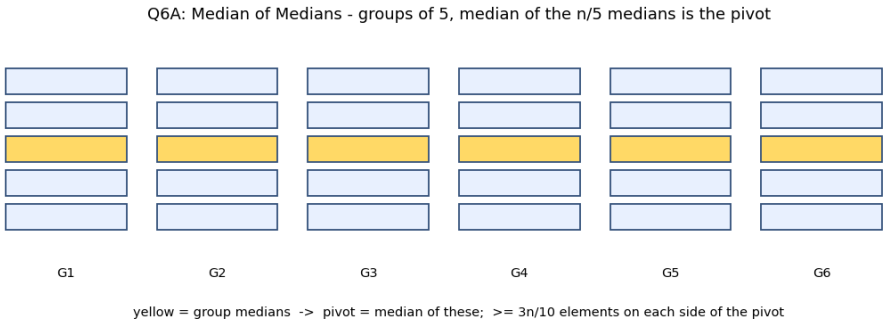


Figure 7.1: Groups of five and the pivot selection

```
SELECT(A, k):
 if |A| <= 5: sort, return A[k]
 medians = median of each group of 5
 p = SELECT(medians, |medians|/2)
 L={x<p}  E={x==p}  G={x>p}
 if k < |L| return SELECT(L,k); if k < |L|+|E| return p; return SELECT(G, k-|L|-|E|)
```

Why the pivot is good: at least half of the n/5 group medians are at most p, and each such group has three elements at most p, so at least 3n/10 elements are on each side. The recursive call therefore has at most 7n/10 + 6 elements. Recurrence: T(n) <= T(n/5) + T(7n/10 + 6) + cn. Since 1/5 + 7/10 = 9/10 < 1, the total is bounded by a geometric series and T(n) = O(n). QuickSelect satisfies T(n) = T(n-1) + n in the worst case (O(n^2)); with random pivots the expectation is O(n) but without a worst-case guarantee.

n (median search)	Input	MoM comparisons	Random-pivot QuickSelect	First-element-pivot QuickSelect
1000	random	7,836	3,669	2,847
1000	sorted	5,538	4,254	375,750
4000	random	32,142	10,316	9,353
4000	sorted	22,950	9,847	6,003,000

Table 7.1: Comparisons to find the median. MoM stays near 5.5n to 8n; the deterministic first-element pivot is quadratic on sorted input (partitions 0 : n-1, depth 2001 for n = 4000).

n	MoM comparisons	comparisons / n	Max recursion depth
1,000	7,519	7.52	10
10,000	81,890	8.19	12
100,000	829,778	8.30	15

Table 7.2: Linear growth of Median of Medians (comparisons per element stay bounded).

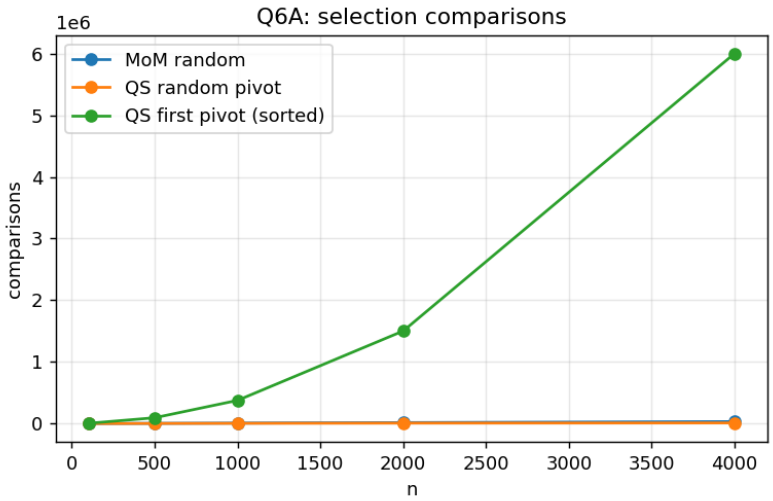


Figure 7.2: Selection comparisons versus n

Observations: randomising the pivot removes the deterministic bad case (8,541 comparisons on the same sorted input, n = 2000), but individual runs vary widely (2,000 to 13,689 over 300 runs). Median of Medians offers a hard bound at a higher constant (about 8 comparisons per element versus about 3.5 for random QuickSelect on random data), so in practice randomised QuickSelect is usually preferred unless the worst-case guarantee matters.

7.2 Meet-in-the-Middle (Student A primary)

Subset Sum: does a subset of n integers add up to T? Brute force examines 2^n subsets. Meet-in-the-Middle divides the items into two halves, enumerates all 2^(n/2) subset sums of each half, sorts them and uses two pointers (one ascending, one descending) to look for a pair summing to T.

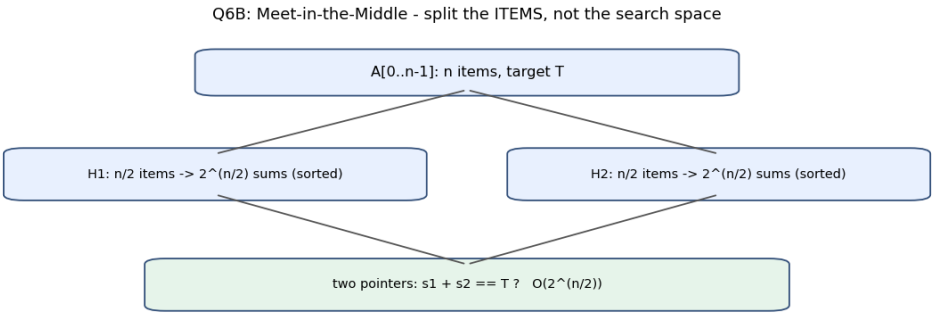


Figure 7.3: Meet-in-the-Middle decomposition

```
MITM(A, T):
 S1 = all subset sums of A[0..n/2-1];  S2 = all subset sums of A[n/2..n-1]
 sort S1 ascending, S2 ascending;  i=0; j=|S2|-1
 while i<|S1| and j>=0:
  s = S1[i]+S2[j]
  if s==T return YES; else if s<T i++ else j--
 return NO
```

Complexity: enumeration and sorting cost O(2^(n/2) * n) time, the two-pointer pass is O(2^(n/2)), and memory is O(2^(n/2)). Brute force needs O(2^n) time and only O(n) memory when streamed. The split divides the items rather than the search space, and the combine step looks up complements instead of testing the product of both halves' choices.

n	Brute: 2^n sums	MITM sums generated	Brute ms	MITM ms
18	262,144	1,024	76.8	0.4
22	4,194,304	4,096	954.0	1.1
24	16,777,216	8,192	4,836.3	1.9
40	1,099,511,627,776 (infeasible)	2,097,152	-	2,000

Table 7.3: Work and time (unreachable target, so both algorithms perform their full work). 400 random small instances agreed with brute force, including reconstruction of an actual subset.

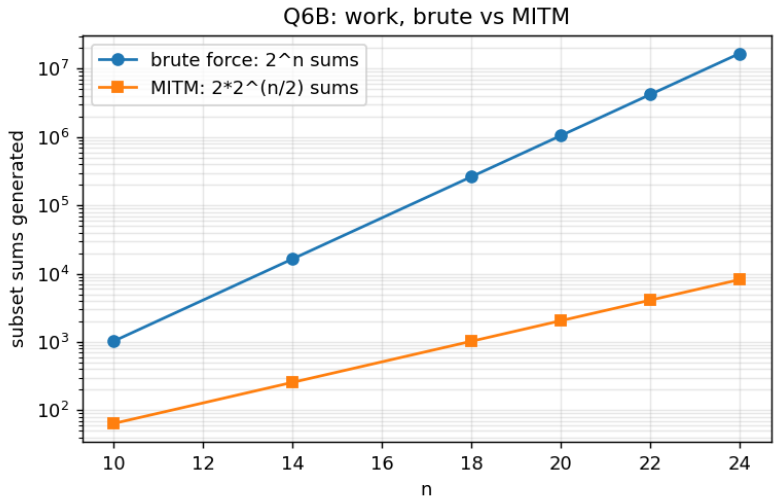


Figure 7.4: Subset sums generated, brute force versus Meet-in-the-Middle

Observations: for n = 40, roughly 2.1 million sums (about 17 MB as 8-byte values, several times more in Python lists) replace 10^12 subsets. The method trades exponentially less time for exponentially more memory than brute force, so memory, not time, becomes the limiting resource at around n = 50 to 60.

7.3 Cross-understanding

- Both algorithms rely on the same principle: restructure the decomposition so that the subproblem sizes or counts shrink geometrically (7n/10 + n/5 in MoM; two sets of 2^(n/2) sums in MITM).

- Median of Medians improves pivot quality at the cost of extra work per level; Meet-in-the-Middle improves time at the cost of memory.

8. Q7: Day-4 Design Challenge

To be completed after the Day-4 problem is released. This section will contain: each student's independent initial design (decomposition, base case, subproblem information, combine strategy), the design comparison, the final joint design, recurrence formulation and solution, correctness tests, baseline comparison and technical conclusion. The corresponding repository folders are Q7_Day4_Design_Challenge/Student_A_Initial_Design, _Initial_Design and Joint_Final_Design.

9. Overall Technical Conclusions

- The value of divide and conquer lies in the decomposition: balanced subproblems (Merge Sort, Closest Pair), fewer subproblems (Karatsuba 4 to 3, Strassen 8 to 7), better pivots (Median of Medians) or splitting the items instead of the search space (Meet-in-the-Middle).
- Recurrences predict operation counts well: every measured count in this report followed its formula, with deviations that could be explained (uneven halves in Q1, extra digits in Q3, padding in Q4).
- Asymptotic advantage does not equal practical advantage. Karatsuba pays off only above hundreds of digits, Strassen did not beat an optimised conventional multiply at any tested size, and brute force wins for tiny closest-pair inputs. Hybrid algorithms with a brute-force base case are the standard remedy.
- Worst-case guarantees cost constants and memory: Median of Medians versus random QuickSelect, Merge Sort's extra space versus Quick Sort, Meet-in-the-Middle's memory versus brute force's O(n).

10. Contribution Summary

Detailed contributions for each question are recorded in Student_Contribution_Record.docx (template in Final_Report/). Student participated in all questions, in the roles specified in the assignment brief.